



Naïve Bayes Email Classification

Group 02 – Jia Khot, Lalit Maurya, Rachit Kumar

Table of contents

01

Dataset Review

02

Training Pipeline

03

Standard Naïve
Bayes

04

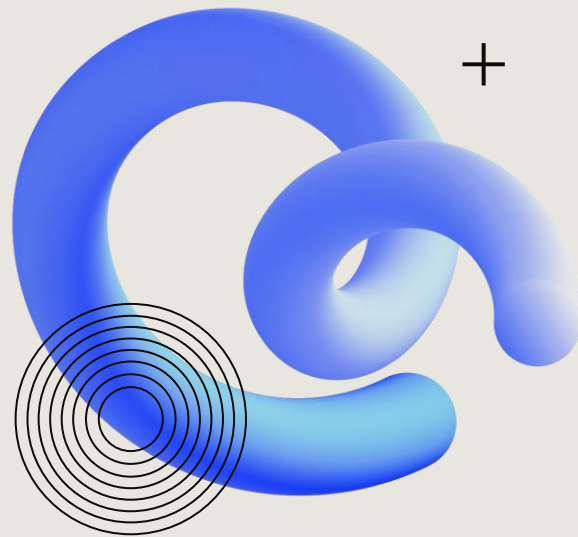
TF-IDFAware
Bayes

05

Evaluation
Metrics

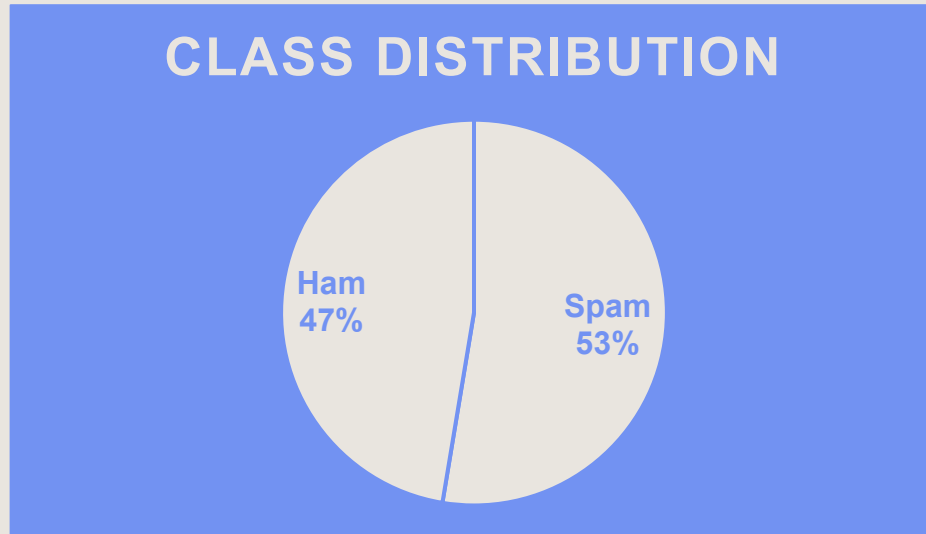
01

Dataset Review



Dataset Review

The dataset used can be found on [Kaggle](#). Having 83448 datapoints. Used in an 80-20 Train-Test split.

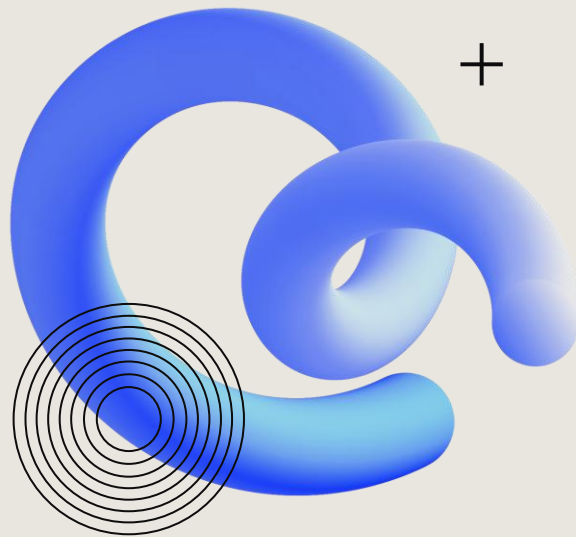


....

+

02

Training Pipeline



Training Pipeline

Preprocessing & Tokenization:

- Convert all text to lowercase and tokenize using a regex-based whitespace tokenizer
- Found to be as effective as `word_tokenize()` from NLTK, while being more computationally efficient
- Removal of stopwords showed no effect on evaluation metrics, however the process does reduce dimensionality, hence we chose to eliminate stopwords from our documents.

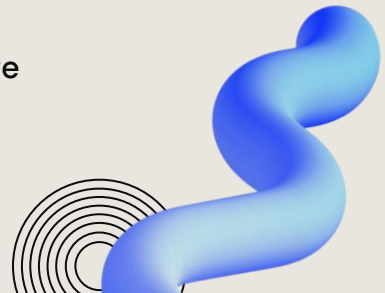
Vocabulary Construction

- Build a vocabulary from the tokenized documents
- Assign a unique integer index to each word (word to index mapping)

Feature Extraction:

- Represent documents using word count features
- Store features as sparse matrices to avoid excessive memory usage (given large dimensionality: $66,758 \times 274,575$)

.....



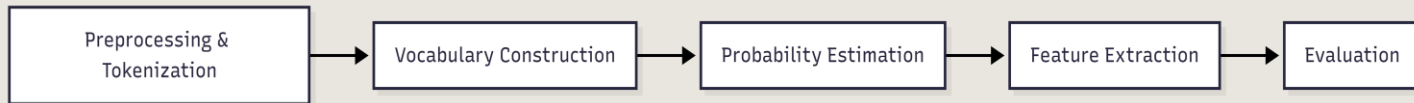
Training Pipeline

Probability Estimation:

- Compute prior probabilities for each class
- Calculate inverse document frequency (IDF) scores for all terms

Model Evaluation

- Evaluate the Naive Bayes model on a held-out test dataset

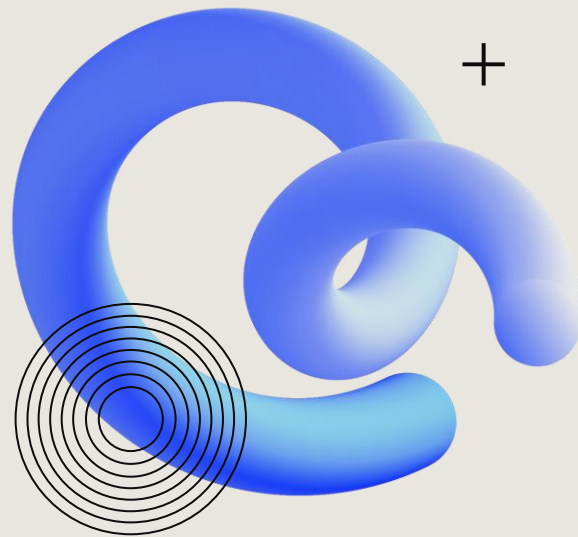


....



03

Standard Naïve Bayes



Standard Naïve Bayes

- In standard Naïve Bayesian classification models, we calculate the prior estimates via simple word counts using the following formula:

$$P(w_i | c) = \frac{\text{Count}(w_i, c) + 1}{|V| + \sum \text{Count}(w_j)}$$

$$\log(P(c|d)) = \log(P(c)) + \sum \log(P(w_i, c))$$

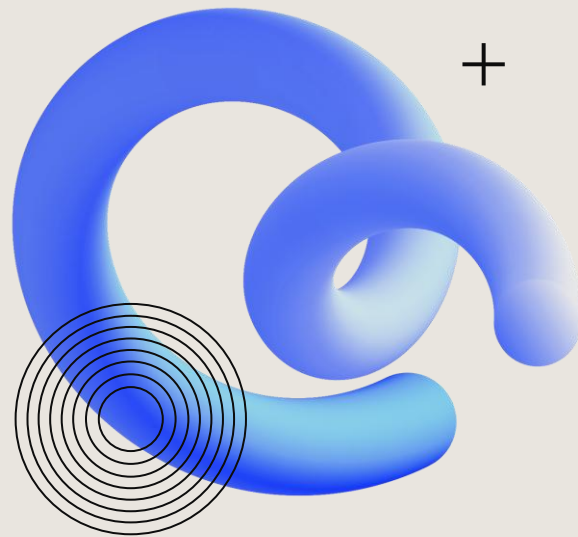
+

....

04

TF-IDF

Aware Bayes



Why TF-IDF?

- Raw word counts overemphasize very common words (e.g., “the”, “is”)
- These frequent terms appear across many documents and provide **low semantic value**
- IDF down-weights such common words and **emphasizes rare, informative terms**
- This improves class separation in Naive Bayes by focusing on meaningful features
- We use the following formula to calculate the TF-IDF score of a word w :

$$TF(w_i, d) = \frac{\text{count}(w_i, d)}{\sum \text{count}(w_j, d)}$$

$$IDF(w) = \log\left(\frac{N}{DF(w) + 1}\right)$$

$$TFIDF(w_i, d) = TF(w_i, d) * IDF(w_i)$$

....

+

How we use TF-IDF Scores?

- The standard formula for probability estimation gets modified to the following version when we include TF-IDF scoring into the mix:

$$\log(P(c | d)) = \log(P(c)) + \sum (\log(P(w_i, c)) * TFIDF(w_i, d))$$

- However, due to inclusion of non-probabilistic terms in the calculation, the result is no longer a valid probability distribution, hence we take a softmax over our log probabilities to normalize our outputs into a valid [0, 1] range such that they sum up to 1.

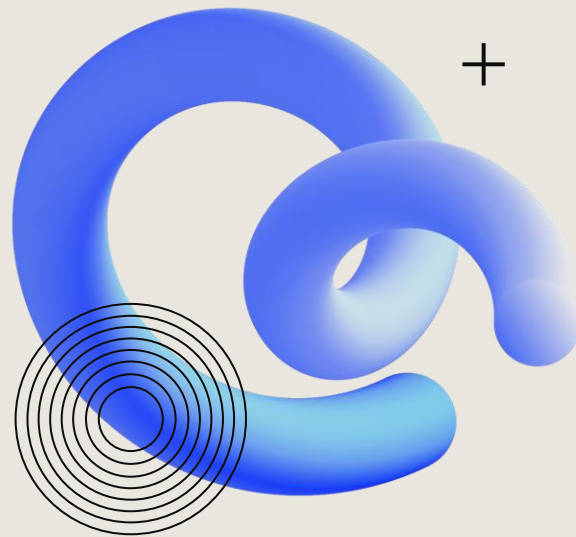
$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

+

....

05

Evaluation Metrics



Evaluation Metrics

Test Set Metrics	Labelled Ham	Labelled Spam
True Ham	8557	137
True Spam	234	7762

Accuracy : 0.9778
Precision : 0.9842
Recall : 0.9734
F1 Score : 0.9788

.....

Train Set Metrics	Labelled Ham	Labelled Spam
True Ham	34708	338
True Spam	411	31301

Accuracy : 0.9888
Precision : 0.9904
Recall : 0.9883
F1 Score : 0.9893

+